

Theoretical Approach toward Understanding Anomaly Detection

Victor Livernoche, Stephen Mikus, Haoran Wang

Comp 562 Final Project - McGill University

Abstract

Anomaly detection plays a crucial role in various domains such as medical monitoring, cybersecurity, fraud detection, and astrology. It is defined as finding unexpected patterns in the data [1]. This paper aims to explore the different types of anomalies, give a definition and understand the underlying distributions. We also explore classical algorithms and deep learning approaches, using autoencoders for detecting anomalies in diverse datasets. Anomaly detection is a field that perfectly demonstrates the no free lunch theorem, as no algorithms consistently outperform others [2]. As more methods are being developed and some are exploiting, in particular, neural networks, understanding and interpreting anomalies is critical to find the best approach to solve this problem. We begin by defining anomalies and discussing their different types, then by presenting the theoretical background for each type and explaining synthetic data generation. The paper then delves into the theoretical foundations of KNN and autoencoder-based methods, stating their strengths and weaknesses in anomaly detection. Lastly, we explain a hybrid model that combines KNN and autoencoder techniques, potentially offering improved performance and versatility in detecting anomalies across various domains.

1 Introduction

Anomaly detection, also known as outlier detection, is the identification of out-of-distribution data based on normal instances, i.e., finding unexpected patterns in the data [1]. It is useful in many domains: medical monitoring, cybersecurity, fraud detection, astrology, etc. [3]. Many algorithms have been developed over the years, adapted from other machine learning tasks, but none outperform others for every dataset; the anomaly detection benchmark perfectly demonstrates the no free lunch theorem [4]. Classical algorithms include clustering and nearest neighbours algorithms, which are simple yet very effective and highly interpretable. K-nearest neighbours [5] is one algorithm that performs well across benchmarks [2]. It is an easily interpretable method that works well by demonstrating a strong anomaly hypothesis. Local Outlier Factor (LOF) [6] is another effective nearest neighbours method that assumes the same hypothesis. Modern approaches include deep learning methods that can generalize for larger data set sizes and high dimensions. Given that anomalies by nature are unknown and thus difficult to define, we begin by explaining what an anomaly means and the different kinds we may encounter. We then present a more theoretical background for the different types of anomalies, how we may generate synthetic versions of each, and give an intuition on how they are defined by analyzing the definitions. Furthermore, we will analyze existing KNN, LOF, and autoencoder methods by first outlining their theoretical base and then exploring their strengths and weaknesses. Finally, we briefly discuss a method that combines both KNN and autoencoders into one hybrid model.

2 Background

There are three kinds of supervision to detect anomalies. Unsupervised and semi-supervised learning does not assume any distribution of anomalies in the data, allowing the detection of unseen anomalies. On the contrary, supervised learning can be reduced to a simple binary classification in which we have a distribution for the anomalies and the normal data. The normal data, as explained in section 3.1, is the set of instances that conform to the expected behavior or is the most common distribution, while anomalies are what we try to detect as they do not fit the distribution of the normal data.

Unsupervised learning. They use the data as it is, without prior training and any label available. Such methods have to make assumptions about the data distribution. This can be challenging for some algorithms as they can easily fit the anomalies within the input data, making identifying them harder. Statistical or clustering methods generally tend to perform better than deep learning methods. This setup is the hardest and the most common.

Semi-supervised learning. We can define two types of semi-supervised setups:

1. Learning from partially labeled data, also called weak supervision, involves using unlabeled data for representation learning and leveraging labeled data to improve detection performance. Algorithms have to be developed for that specific setup.
2. Using only normal samples for training to identify samples that deviate from the learned model as anomalous. Unsupervised algorithms can be applied to this setup to learn the normal distribution only. At test time, the anomalies will be different from the learned distribution.

Supervised learning. This reduces the anomaly detection problem to supervised classification, usually with unbalanced classes, since the number of anomalies in the data is generally a fraction of the total number of samples. This setup is less significant since such algorithms can only identify known anomalies while failing to discover unknown anomalies within the data.

As unsupervised algorithms do not rely on any assumption on the anomaly distribution, we will focus on them from now on as they are the most suitable for this problem. This also applies to the semi-supervised setup with normal samples only for training.

Types of anomalies We can also consider different types of anomalies. They are defined by three categories [1][2][7]:

1. **Point anomalies.** This is the simplest category for anomalies; it is when a single instance of the data is considered anomalous with respect to the rest of the data set. For example, if a patient's blood pressure reading is significantly higher or lower than others in a clinical data set, that data point would be considered a point anomaly. We can define **local** anomalies to be points that are deviant from their local neighbourhoods and **global** anomalies to be points that are more different from the overall normal data.
2. **Clustered anomalies.** These are instances that are anomalous as a set compared to the rest of the dataset; the individual points are not necessarily anomalous by themselves. They are also known as **group** anomalies. For example, multiple people connecting on a network simultaneously more than on occasion should be flagged as an anomaly.
3. **Contextual anomalies.** These are anomalies that are only anomalous in a specific context. For example, the number of visits to an e-commerce store spiking during holiday sales should not be considered anomalous, but if we had the same spike during a normal day, it should be considered an anomaly.

Most algorithms give an anomaly score for each instance of the data set; the higher the score is, the more anomalous the instance is. We can choose a threshold based on the desired trade-off between false positives and false negatives. This is the case for clustering and nearest neighbour algorithms, which both use distance to score data instances, which makes them easily interpretable. Clustering algorithms assume that the anomalies are either not part of any cluster or are in smaller clusters than normal instances, or lie further away from the cluster centroid [8]. They work by grouping similar data instances into clusters and then applying one of the assumptions on each instance to find an anomaly score. They include CBLOF [9], which after clustering the data, calculates the size of the cluster in which a data instance belongs to find its score, and k-means [10], which works by partitioning the instances into k sets to minimize the sum of squares within the cluster. Nearest neighbour algorithms use the distance between data instances or their relative density with respect to their local neighbourhood. K-nearest neighbours [5] is still one of the most widely used detection

algorithms, as it performs well on most data sets. LOF [6] and COF [11] are other nearest neighbour-based algorithm that assigns an anomaly score based on the ratio of the average local density of the k -nearest neighbour and the local density of an instance. Other nearest neighbour-based algorithms include variants of LOF, such as LOCI [12].

As anomalies can be unnoticeable in a high-dimensional space and are more complex [13], the development of deep anomaly detection algorithms have been increasing over the past few years [7], as they are better suited to handle these issues. Autoencoders have been used with a combination of other classical methods to detect anomalies [14][15][16][17][18][19][20]. These works but leveraging the capacity of autoencoders to learn the probability distribution of a data set by using the inputs as targets [21]. Other works include DeepSVDD [22], a method that learns a hypersphere that encloses the normal data, and DAGMM uses an autoencoder to generate a representation fed into a Gaussian Mixture Model [23]. More recently, ICL [24], which uses internal contrastive loss on pairs of partition of features from the input instances, achieve state-of-the-art results for ODDS data sets for the semi-supervised setting with normal samples for training.

3 Theoretical analysis

3.1 Understanding Anomalies

3.1.1 Definition

1. **Point anomaly.** A point anomaly in a data set $\mathcal{D}$ is an individual data instance $x_i \in \mathcal{D}$ that significantly deviates from the expected distribution of the data set. More specifically, if we define the distribution of the data set as $P(\mathcal{D})$ and the distribution of an individual data instance as $P(x_i)$, then a point anomaly is a data instance x_i for which:

$$P(x_i) \ll P(\mathcal{D})$$

This means that the probability of observing the data instance x_i under the expected distribution $P(\mathcal{D})$ is very low compared to the probability of observing other data instances in the dataset. For example, in the case of a 1D dataset drawn from a normal distribution, $X \sim \mathcal{N}(\mu, \sigma)$, we can calculate the probability of observing a given data instance x_i using the probability density function (PDF) of the normal distribution:

$$P(x_i) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x_i - \mu)^2}{2\sigma^2}} \quad (1)$$

where μ and σ are the mean and standard deviation of the normal distribution, respectively. To identify point anomalies in this dataset, we can set a threshold value η such that any data instance with $P(x_i) < \eta$ is considered a point anomaly. A simple way to set the threshold, as we have a normal distribution, is to set a threshold based on the probability of a data point appearing outside of a specific standard deviation, say, for example, if $x_i > 2\sigma$ or $x_i < -2\sigma$. These data instances have low probabilities of occurring under the normal distribution, indicating that they are significantly different from the rest of the dataset and could be considered point anomalies.

2. **Group Anomaly.** Given a data set $\mathcal{D}$, a subset of data instances $\mathcal{G} \subset \mathcal{D}$ is considered a group anomaly if the probability of observing the group $\mathcal{G}$ under the expected distribution of the dataset, $P(\mathcal{D})$, is significantly lower compared to the probability of observing other subsets of data instances in the dataset. Mathematically, a group anomaly is defined as:

$$\mathcal{G} \subset \mathcal{D} \quad \text{such that} \quad P(\mathcal{G}) \ll P(\mathcal{D})$$

where $P(\mathcal{G})$ is the probability of observing the group $\mathcal{G}$ under the expected distribution, and $P(\mathcal{D})$ is the probability of observing the entire dataset $\mathcal{D}$ under the same distribution. To identify group anomalies, a threshold value η can be set such that any group of data instances with $P(\mathcal{G}) < \eta$ is considered a group anomaly.

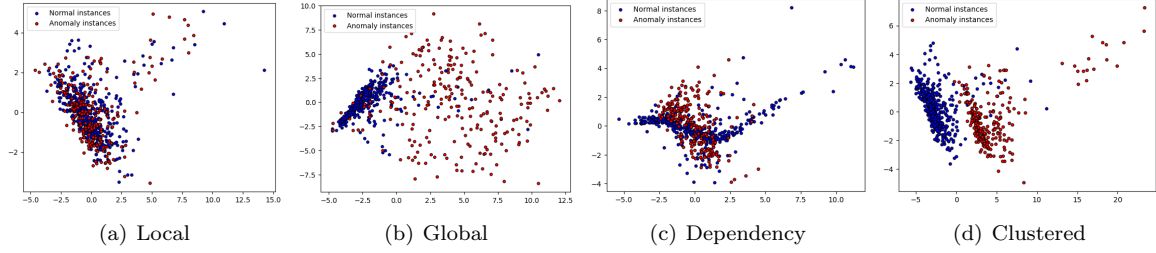


Figure 1: Plot of the four different types of anomalies on the [breastw](#) dataset with PCA dimensionality reduction generated by the procedure described in section 3.1.2.

3. **Contextual Anomaly.** Given a data set $\mathcal{D}$ and a set of contextual attributes $\mathcal{C}$, a data instance $x_i \in \mathcal{D}$ is considered a contextual anomaly if its value significantly deviates from the expected value of data instances with similar contextual attributes. Mathematically, a contextual anomaly is defined as:

$$x_i \in \mathcal{D} \quad \text{such that} \quad P(x_i|\mathcal{C}) \ll P(\mathcal{D}|\mathcal{C})$$

where $P(x_i|\mathcal{C})$ is the conditional probability of observing the data instance x_i given the contextual attributes $\mathcal{C}$, and $P(\mathcal{D}|\mathcal{C})$ is the conditional probability of observing the entire dataset $\mathcal{D}$ given the same contextual attributes. To identify contextual anomalies, a threshold value η can be set such that any data instance with $P(x_i|\mathcal{C}) < \eta$ is considered a contextual anomaly.

3.1.2 Synthetic Anomalies

There has been a few methods for generating synthetic anomalies developed over the years [25][26]. It is interesting to consider them as they give an interpretation of what anomalies are. We can then better understand anomalies and do a mathematical analysis based on the synthesized data. We consider a dataset $\mathcal{D}$ with labels y , where $y = 0$ corresponds to normal data and $y = 1$ corresponds to anomalies. For the four types of synthetic outliers: global outliers, dependency outliers, and cluster outliers, we first fit a generative model on the normal samples of X to then recreate normal data from this model and modify it to create anomalies.

1. **Local anomalies:** To create local outliers, we first fit a Gaussian Mixture Model (GMM) on the normal data. We select the optimal number of components by minimizing the Bayesian Information Criterion (BIC). The GMM is given by:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x; \mu_k, \Sigma_k), \quad (2)$$

where K is the number of components, π_k are the mixing coefficients, and $\mathcal{N}(x; \mu_k, \Sigma_k)$ are the Gaussian distributions with means μ_k and covariance matrices Σ_k . To generate local outliers, we modify the GMM's covariance matrices by multiplying them with a scaling factor $\alpha > 1$:

$$\Sigma'_k = \alpha \Sigma_k,$$

and then sample synthetic anomalous data from the modified GMM.

2. **Global anomalies:** In this case, we again fit a GMM on the normal data. However, instead of modifying the GMM parameters, we generate synthetic anomalous data by sampling from a uniform distribution with the following bounds:

$$a_i = \alpha \cdot \min(X_{:,i}),$$

$$b_i = \alpha \cdot \max(X_{:,i}),$$

where i refers to the i^{th} feature of the dataset and $\alpha > 1$. For each feature i , $X_i \sim U(a_i, b_i)$.

3. **Dependency anomalies:** To generate dependency outliers, we first fit a vine copula model on the normal data. A vine copula is a high-dimensional copula that models the dependencies between variables using a sequence of bivariate copulas. We use the C-vine copula by default. After fitting the copula model, we sample synthetic normal data from it. To generate synthetic anomalous data, we independently sample each feature using a Gaussian Kernel Density Estimator (KDE). The synthetic anomalous data is then formed by concatenating these independently sampled features.
4. **Clustered anomalies:** Similar to local outliers, we start by fitting a GMM on the normal data. In this case, we modify the means of the Gaussian distributions by multiplying them with a scaling factor $\alpha > 1$:

$$\mu'_k = \alpha \mu_k.$$

Finally, we sample synthetic anomalous data from the GMM with the modified means. The synthetic normal and anomalous data are combined to form the final dataset with the corresponding labels.

For local anomalies, let's analyze what it means for an instance to be an anomaly. Let's take the base case with the number of components $K = 1$ since there is no way to simplify with more components. Then, we have $p(x)$, the probability distribution of the normal samples and $q(x)$, the probability distribution of the anomalies. Let's look at the KL divergence between these distributions.

$$D_{KL}(p||q) = \int p(x) \log \frac{p(x)}{q(x)} dx \quad (3)$$

Let's use the defined distributions from synthetic anomalies in Equation 2.

$$D_{KL}(p||q) = \int p(x) \log \frac{\mathcal{N}(x; \mu, \Sigma)}{\mathcal{N}(x; \mu, \alpha \Sigma)} dx$$

Now, let's substitute the expressions for the Gaussian distributions and simplify:

$$\begin{aligned} D_{KL}(p||q) &= \int p(x) \log \frac{\frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}}{\frac{1}{(2\pi)^{d/2} |\alpha \Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T (\alpha \Sigma)^{-1}(x-\mu)}} dx \\ &= \int p(x) \log \frac{|\alpha \Sigma|^{1/2} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}}{|\Sigma|^{1/2} e^{-\frac{1}{2}(x-\mu)^T (\alpha \Sigma)^{-1}(x-\mu)}} dx \\ &= \int p(x) \log \frac{|\alpha \Sigma|^{1/2} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}}{|\Sigma|^{1/2} e^{-\frac{1}{2}(x-\mu)^T (\alpha \Sigma)^{-1}(x-\mu)}} dx \\ &= \int p(x) \log \frac{\alpha^{d/2} |\Sigma|^{1/2} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}}{|\Sigma|^{1/2} e^{-\frac{1}{2\alpha}(x-\mu)^T \Sigma^{-1}(x-\mu)}} dx \\ &= \int p(x) \left(\log \alpha^{d/2} + \frac{\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}{\frac{1}{2\alpha}(x-\mu)^T \Sigma^{-1}(x-\mu)} \right) dx \\ &= \int p(x) \left(\log \alpha^{d/2} + \alpha \right) dx \\ &= \log \alpha^{d/2} + \alpha \end{aligned} \quad (4)$$

Intuitively, the KL divergence measures the amount of information lost when using the distribution $q(x)$ to approximate $p(x)$. It shows that, in a sense, local anomalies can be seen as a loss in information from the true distribution of the normal data. The term $\log(\alpha^{d/2})$ is related to the dimensionality of the dataset d . As the dimensionality increases, the logarithm of α raised to the power of $d/2$ will also increase. This implies that, in high-dimensional datasets, the divergence between the normal data and anomalies becomes larger, which can make it easier to distinguish between them, but more information is lost. The term α is a constant, and it represents an overall shift in the KL divergence between the two distributions. In general, a larger α indicates a greater divergence between the normal data and the anomalies. This term can be influenced by the specific characteristics of the dataset, such as how well-separated the normal data and anomalies are.

Similarly, for clustered anomalies, we can find the KL divergence [27], and we get :

$$\begin{aligned}
D_{KL}(p||q) &= \int p(x) \log \frac{p(x)}{q(x)} dx \\
&= \int p(x) \log \frac{\mathcal{N}(x; \mu, \Sigma)}{\mathcal{N}(x; \alpha\mu, \Sigma)} dx \\
&= \frac{1}{2} (\alpha\mu - \mu)^T \Sigma^{-1} (\alpha\mu - \mu) \\
&= \frac{1}{2} (\alpha - 1)^2 \mu^T \Sigma^{-1} \mu
\end{aligned} \tag{5}$$

Here, increasing alpha increases the information lost, as expected. Interestingly, the divergence depends on the mean and the covariance matrix of the distribution. That makes sense since the clustered anomaly generation directly depends on the main cluster distribution. For global anomalies, we basically have just random noise since we are sampling from the uniform distribution. For the dependency anomalies, the analysis is harder and would require a more in-depth search.

3.1.3 Formal Learning Definition

Given a dataset $\mathcal{D} = x_1, x_2, \dots, x_n$, where each data point x_i belongs to a feature space $\mathcal{X}$, the goal of anomaly detection is to find a subset of data points $\mathcal{A} \subseteq \mathcal{D}$, such that $\mathcal{A}$ contains the anomalous instances. The dataset can be partitioned into two classes: normal instances $\mathcal{X}_N$ and anomalous instances $\mathcal{X}_A$. The problem consists of identifying the instances in $\mathcal{X}_A$.

Typically, the dataset is characterized by a significant class imbalance, with the majority of instances belonging to the normal class $\mathcal{X}_N$, and a small fraction belonging to the anomalous class $\mathcal{X}_A$. The primary challenge in anomaly detection is to effectively distinguish the rare anomalous instances from the normal instances, despite the class imbalance. It can also be seen as a bipartite ranking.

Given a hypothesis set H of functions mapping X to $\mathbb{R}^n$, the learning problem in anomaly detection consists of selecting a hypothesis $h \in H$ that can effectively distinguish between normal instances $\mathcal{X}_N$ and anomalous instances $\mathcal{X}_A$. To formalize this, let's first define the generalization error $R(h)$ and the empirical error $\hat{R}_{\mathcal{D}}(h)$. The generalization error $R(h)$ measures the expected probability of misclassification between the normal and anomalous instances under the hypothesis h . Mathematically, it can be expressed as:

$$R(h) = \mathbb{P}_{x \sim \mathcal{X}_N, x' \sim \mathcal{X}_A} [h(x) > h(x')] \tag{6}$$

The empirical error $\hat{R}_{\mathcal{D}}(h)$ is computed based on the dataset $\mathcal{D}$ and represents the fraction of misclassified instances in the dataset under the hypothesis h . It can be formulated as follows:

$$\hat{R}_{\mathcal{D}}(h) = \frac{1}{|\mathcal{X}_N| |\mathcal{X}_A|} \sum_{x \in \mathcal{X}_N} \sum_{x' \in \mathcal{X}_A} 1_{h(x) > h(x')} \tag{7}$$

In the context of anomaly detection, the objective is to find a hypothesis $h^* \in H$ that minimizes the generalization error $R(h)$:

$$h^* = \arg \min_{h \in H} R_{\mathcal{D}}(h) \tag{8}$$

However, since the true generalization error $R(h)$ is not accessible in practice, supervised learning algorithms need to minimize the empirical error $\hat{R}_{\mathcal{D}}(h)$:

$$\hat{h} = \arg \min_{h \in H} \hat{R}_{\mathcal{D}}(h) \tag{9}$$

By minimizing the empirical error, the learning algorithm aims to find a hypothesis $\hat{h}$ that effectively distinguishes between normal and anomalous instances in the dataset, despite the class imbalance. The issue is that labels are not available in the unsupervised setting; thus, the empirical error is also not available. We have to assume an underlying property of the data that distinguishes the anomalies and the normal instances to come up with an unsupervised anomaly detection method. The goal is

to minimize the misclassification probability, i.e., the likelihood of classifying a normal instance as anomalous or vice versa. We can assign a score for each element of the dataset $\mathcal{D}$, which ranks each instance in terms of how anomalous they are. We can then either set a threshold to pick the η most anomalous samples or evaluate the performance with the AUC ROC, AUC PR or F1 score (needs a threshold), to name a few evaluation metrics.

3.2 K-nearest neighbours

K-Nearest neighbours is one of the most simple machine learning algorithms. For a given data point and labelled data set, it will predict the label for the data point based on its distance from its K-nearest neighbouring samples. The closer a sample is to another, the more similar they are to each other. Similarly, the farther away a sample is from another, the more they are unlike each other. This forms the basis of outlier detection using KNN.

This algorithm works under the assumption that normal instances belong in dense neighbourhoods, while anomalies occur far from their closest neighbours. Since it is still one of the best-performing baselines in the anomaly detection literature, this shows that this simple assumption is accurate in some sense. It also follows what was previously shown in section 3.1.2.

We define $\mathcal{D} \in \mathcal{R}^d$ to be a d -dimensional data set consisting of unlabelled samples x_i for $i \in 1, \dots, n$. Let n_x^k be the distance between points x and its k -nearest neighbour where the distance measurement can be calculated using any $\mathcal{L}_p$ metric (ex. Manhattan ($\mathcal{L}_1$), Euclidean ($\mathcal{L}_2$), etc.). We defined the most commonly used distance, the Euclidean distance, as follows:

$$D(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{i=1}^d (x^{(i)} - x'^{(i)})^2} \quad (10)$$

where $x^{(i)}$ and $x'^{(i)}$ are the i^{th} feature of $\mathbf{x}$ and $\mathbf{x}'$, respectively.

In the high dimensional case, one may consider using fractional distance metrics ($p \in (0, 1)$) rather than the standard p norms. Larger p values place undue importance on more sparse dimensions, which are dominated by larger distances leading to poorer results [28]. It is important to note that while the selection of the metric will impact the measurements, we will leave the distance function unspecified for the sake of generality.

The first method we will cover is based solely on the distance from a point to its k^{th} nearest neighbour. The process of determining outliers is straightforward: given a threshold of η points, and a parameter k , the number of neighbours to consider, we classify a point x as anomalous if there exist no more than $\eta - 1$ points x' such that $n_{x'}^k > n_x^k$. For instance, if $\eta = 5$ x is classified as an anomaly if there are no more than 4 other points in the dataset such that the distance from them to their k^{th} nearest neighbour is larger. While straightforward, the presence of outliers in the data set and the selection of η is critical for this approach. In the absence of errant data, points classified as outliers may have values of n_x^k that are only fractionally larger than the rest of the data set. For example, if we consider the decreasing set $\{1.5, 1.49, 1.48, 1.47, \dots\}$ where each value represents $n_{x_i}^k$, with $i \in [1, \dots, n]$, then we may consider the first η elements to be anomalous though they only have slightly larger k^{th} neighbour distances than the rest of the data. Similarly, if we choose η to be too large or too small, then we risk overestimating and underestimating the number of outliers, respectively. For benchmark evaluation, we can use the area under the curve to avoid the issue of selecting a threshold value η .

Since the true η is unknown, another method of outlier detection that relies on calculating the distances from a given point x to all of its k -nearest neighbours can be used [29]. For this method, we set a threshold distance ϵ instead of a threshold for the number of points. We define:

$$\mathcal{N}_x^k = \{x' \in \mathcal{D} | dist(x, x') \leq \epsilon \wedge x' \neq x\} \quad (11)$$

This denotes the set of all the k -nearest neighbours to the point x . Now, for some $\epsilon > 0$ we consider a point x to be an outlier if $\sum_{x' \in \mathcal{N}_x^k} \text{dist}(x, x') > \epsilon$. In other words, if the sum of all distances from x to its k -nearest neighbours exceeds some threshold value, then we may consider it to be an anomalous point.

3.3 Local Outlier Factor

This method works under the same assumption as KNN (see section 3.2). Local Outlier Factor (LOF) is another intricate method that relies on scoring data points based on local density. Using the definitions from the standard KNN algorithm, we define the Reachability Distance:

$$\mathcal{R}_d(x, x') = \max\{\text{dist}(x, x'), n_x^k\} \quad (12)$$

Points that lie within a distance n_x^k from a point x have all their distances taken to be equal, while points that lie outside this range keep their true distances. This helps reduce the oscillation within points that lie in the same neighbourhood [6]. We now define Local Reachability Distance:

$$\text{lrd}(x) = \left(\frac{\sum_{x' \in \mathcal{N}_x^k} \mathcal{R}_d(x, x')}{|\mathcal{N}_x^k|} \right)^{-1} \quad (13)$$

to be the inverse of the average reachability of point x to x' . This value can become ∞ in the case that duplicate values are present in the given reachability distance [6]. Incorporating everything thus far, we can define the Local Outlier Factor:

$$\text{LOF}_k(x) = \frac{\sum_{x' \in \mathcal{N}_x^k} \frac{\text{lrd}(x')}{\text{lrd}(x)}}{|\mathcal{N}_x^k|} = \frac{\sum_{x' \in \mathcal{N}_x^k} \text{lrd}(x')}{|\mathcal{N}_x^k| \cdot \text{lrd}(x)} \quad (14)$$

This value is a comparison of the average local reachability density of the k neighbourhood of x compared to that of all its neighbours. We can say that, for some x , if:

1. $\text{LOF}(x) \approx 1$ then its density is almost equal to that of its neighbours.
2. $\text{LOF}(x) < 1$ then x has a higher density than its neighbours and is, therefore, closer to the center of the neighbourhood.
3. $\text{LOF}(x) > 1$ then the density of x is lower than its k neighbourhood's density and is then considered to be an outlier.

Local Outlier Factor has the advantage of using the relative density of points compared to the density of all of the points in its k neighbourhood rather than just a simple threshold. This also helps when data sets have very dense/sparse clusters of samples where a small/large deviation would indicate an outlier, solving the problem from the previous methods.

3.4 Autoencoders

Autoencoders, a specific type of feedforward Neural Network, are designed to learn useful features by encoding the input into compressed representations and decoding it back so that the reconstructed input is as similar to the original data as possible [30]. Some datasets with unique data characteristics, high data dimensions, and noises can be hard to process, and autoencoders could be useful in processing them. As we focus on unsupervised anomaly detection, where the goal is, given the normal data examples, to learn a normal profile and then identify the example as an anomaly if the example does not correspond to the normal profile. By assumption, autoencoders are considered an unsupervised learning technique, and the trained autoencoder learns the latent subspace of normal samples. After training, the autoencoders would give low reconstruction errors for normal examples, whereas high reconstruction errors for anomalies [30][31].

3.4.1 Architecture

Autoencoders are composed of two neural networks (NN); A and B work as encoder and decoder, which are defined by $\mathbf{A} : \mathbb{R}^n \rightarrow \mathbb{R}^p$ and $\mathbf{B} : \mathbb{R}^p \rightarrow \mathbb{R}^n$ [31]:

$$\mathbf{A} = \sigma(\mathbf{W}_{xh}x + b_{xh})$$

$$\mathbf{B} = \sigma(\mathbf{W}_{hx}x + b_{hx})$$

where encoder A maps an input vector x to an informative representation and decoder B maps the hidden representation back to the original space. σ here is the activation function, $\mathbf{W}$ is the weight and b is the bias of neural networks[17]. $(\mathbf{W}, b)$ as the parameters of the NN, which can be learned through the training process by minimizing the reconstruction error:

$$(\mathbf{W}_{xh}, b_{xh}), (\mathbf{W}_{hx}, b_{hx}) = \arg \min_{A, B} \mathbf{E}[\|x, \mathbf{B} \circ \mathbf{A}(x)\|] \quad (15)$$

In some special cases, A and B could be linear operations where we get a linear Autoencoder. By dropping the non-linear operations, Autoencoders result in the same latent data representations as PCA [32]. However, Autoencoders with non-linear activation functions are able to capture the anomalies that PCA fails to detect, as many features are non-linearly correlated. It is also more efficient for Autoencoders to train several layers instead of training one complex transformation in PCA [14][19].

3.4.2 Types of Autoencoders

1. **Conventional Autoencoder.** Follows the structure above, consisting of an encoder and a decoder. The encoder compresses the input samples into a smaller number of neurons, a hidden layer, and then activated by sigmoid in our case. Decoder will transfer the deep representation back into the original input space $\mathbb{R}^d$. Optimizing the construction errors will result in the weights $\mathbf{W}$ and bias b .
2. **Variational Autoencode.** Autoencoders have a probabilistic distribution built in, which will entitle them to not only compute but give an estimation of the sample data. It is similar to the conventional Autoencoders in regard to the encoding process. However, after the hidden layer, a probabilistic model is added. Thus, during decoding, the reconstruction of the sample can be realized by the probabilistic model built in [33].
3. **Convolutional Autoencoder.** Convolutional Autoencoders (CAE) contain convolutional neural networks as encoders and decoders. By the nature of convolutional layers, data first needs to be transformed into 2D before feeding into the CNN. Here, the activation function still needs to be non-linear (e.g. sigmoid):

$$h_l^k = \sigma\left(\sum_{j \in J} x_l^j \otimes w_l^k + b^k\right) \quad (16)$$

where $\otimes$ is the 2D convolutional operator. Similarly, the deconvolution layer will recover the lower dimension hidden layer back to the original space [34].

3.4.3 Methodology of Autoencoder-Based Anomaly Detection

Autoencoder Training. In section 3.1, we defined ϵ as the threshold. For training purposes, we let the reconstruction errors ϵ_i as anomaly scores. For the original dataset $\mathcal{D} = ((\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n))$, we have the reproduced dataset $\mathcal{D}' = (\mathbf{x}'_1, \mathbf{x}'_2, \dots, \mathbf{x}'_n)$:

$$\epsilon(\mathbf{x}_i, \mathbf{x}'_i) = \sum_{j=1}^d \|\mathbf{x}_i - \mathbf{x}'_i\|^2 \quad (17)$$

By minimizing the reconstruction error, we have $(\mathbf{W}_{xh}^*, b_{xh}^*)$ and $(\mathbf{W}_{hx}^*, b_{hx}^*)$ as the minimizer to be the parameters of encoder and decoder:

$$(\mathbf{W}_{xh}^*, b_{xh}^*), (\mathbf{W}_{hx}^*, b_{hx}^*) = \arg \min_{(\mathbf{W}_{xh}, b_{xh}), (\mathbf{W}_{hx}, b_{hx})} \frac{1}{N} \sum_{i=1}^n \epsilon(\mathbf{x}_i, \mathbf{x}'_i) \quad (18)$$

Autoencoder Testing. We feed the testing data into the trained Autoencoders to get the reconstruction error from Equation 17, which can be seen as an anomaly score. We then define a classifier as in Section 3.1 as:

$$c(x_i) = \begin{cases} normal & \text{if } \epsilon_i < \epsilon \\ anomaly & \text{if } \epsilon_i > \epsilon \end{cases}$$

According to the classifier, anomalies will have higher anomaly scores, which are the reconstruction errors, than the normal samples.¹

3.4.4 Autoencoders with KNN (AEKNN)

KNN, as described in section 3.2, is a well-known classification algorithm. However, it tends to perform worse as the number of samples or the dimension of the samples grows. By lowering the dimension of the samples through Autoencoders to have a new and more informative deep representation of data, KNN will have better performance for anomaly detection. This Autoencoder KNN-based classifier with built-in dimension reduction is often referred to as AEKNN.

In this integrated algorithm, the framework also contains training and testing two steps. Training Autoencoders: we first train the Autoencoders as shown in the section 3.4.3. By minimizing reconstruction errors, we can get the parameters of the encoder and decoder.

With the trained autoencoder, in contrast to the testing part in autoencoder or classical KNN, we first feed the data into the trained autoencoder. After encoding the input data, we can get the most compressed input data with lower dimensions. This hidden deep representation of the input data vector will be the input data for the integrated KNN model, as this compressed vector will better manifest the differences between normal data and anomalies. The main function of KNN here is to distinguish the normal samples from the outliers by computing the distances. The KNN anomaly detection model will utilize the distance between data points and k^{th} nearest neighbours and then give a list with top n as anomalies [36][37].

4 Conclusion

KNN, LOF, and autoencoders exhibit unique characteristics that make them suitable for specific types of anomaly detection tasks. The KNN algorithm, with its simplicity and ease of implementation, is a powerful tool for detecting local anomalies based on the proximity of data points in feature space. Its effectiveness in cases where the notion of similarity is well-defined and interpretable makes it an attractive choice for many applications. However, KNN’s performance, similarly to LOF’s, may deteriorate in high-dimensional spaces and with noisy or imbalanced datasets.

On the other hand, autoencoders have emerged as an approach for anomaly detection where the data exhibits complex structures or relationships. By leveraging their ability to learn latent representations, autoencoders can effectively capture the underlying structure of the data, allowing for the identification of anomalies that deviate from this structure. Their flexibility in handling various data types, including unstructured data such as images and text, further broadens their applicability. Despite their advantages, autoencoders may require more computational resources and hyperparameter tuning to achieve optimal performance, which could be a limiting factor in certain use cases. It can also more easily overfit the anomalies in the dataset.

In this paper, we have delved into the challenging task of anomaly detection. We began by providing a comprehensive introduction to the concept of anomalies, analyzed the different types and defined synthetic anomalies and learning approaches. We then presented an analysis of KNN, LOF, and autoencoders, highlighting their underlying principles, some strengths and some limitations.

¹Note: For convolutional Autoencoders, we need to pre-process the data into 2D data as the convolutional neural networks only take 2D data types. Multiple methods can apply here, such as triangle Area Maps [14][35].

References

- [1] A. B. Nassif, M. A. Talib, Q. Nasir, and F. M. Dakalbab, “Machine learning for anomaly detection: A systematic review,” *IEEE Access*, vol. 9, pp. 78 658–78 700, 2021.
- [2] S. Han, X. Hu, H. Huang, M. Jiang, and Y. Zhao, “Adbench: Anomaly detection benchmark,” 2022. [Online]. Available: <https://arxiv.org/abs/2206.09426>
- [3] V. Hodge and J. Austin, “A survey of outlier detection methodologies,” *Artificial Intelligence Review*, vol. 22, pp. 85–126, 10 2004.
- [4] D. Wolpert and W. Macready, “No free lunch theorems for optimization,” *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.
- [5] S. Ramaswamy, R. Rastogi, and K. Shim, “Efficient algorithms for mining outliers from large data sets,” *SIGMOD Rec.*, vol. 29, no. 2, p. 427–438, may 2000. [Online]. Available: <https://doi.org/10.1145/335191.335437>
- [6] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “Lof: Identifying density-based local outliers,” in *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’00. New York, NY, USA: Association for Computing Machinery, 2000, p. 93–104. [Online]. Available: <https://doi.org/10.1145/342009.335388>
- [7] L. Ruff, J. R. Kauffmann, R. A. Vandermeulen, G. Montavon, W. Samek, M. Kloft, T. G. Dietterich, and K.-R. Muller, “A unifying review of deep and shallow anomaly detection,” *Proceedings of the IEEE*, vol. 109, no. 5, pp. 756–795, may 2021. [Online]. Available: <https://doi.org/10.1109%2Fjproc.2021.3052449>
- [8] V. Chandola and V. Kumar, “Outlier detection : A survey,” *ACM Computing Surveys*, vol. 41, 01 2009.
- [9] Z. He, X. Xu, and S. Deng, “Discovering cluster-based local outliers,” *Pattern Recogn. Lett.*, vol. 24, no. 9–10, p. 1641–1650, jun 2003. [Online]. Available: [https://doi.org/10.1016/S0167-8655\(03\)00003-5](https://doi.org/10.1016/S0167-8655(03)00003-5)
- [10] J. B. MacQueen, “Some methods for classification and analysis of multivariate observations,” in *Proc. of the fifth Berkeley Symposium on Mathematical Statistics and Probability*, L. M. L. Cam and J. Neyman, Eds., vol. 1. University of California Press, 1967, pp. 281–297.
- [11] J. Tang, Z. Chen, A. W.-C. Fu, and D. W.-L. Cheung, “Enhancing effectiveness of outlier detections for low density patterns,” in *Proceedings of the 6th Pacific-Asia Conference on Advances in Knowledge Discovery and Data Mining*, ser. PAKDD ’02. Berlin, Heidelberg: Springer-Verlag, 2002, p. 535–548.
- [12] S. Papadimitriou, H. Kitagawa, P. Gibbons, and C. Faloutsos, “Loci: Fast outlier detection using the local correlation integral.” 01 2003, pp. 315–326.
- [13] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, “Deep learning for anomaly detection,” *ACM Computing Surveys*, vol. 54, no. 2, pp. 1–38, mar 2021. [Online]. Available: <https://doi.org/10.1145%2F3439950>
- [14] Z. Chen, C. K. Yeo, B. S. Lee, and C. T. Lau, “Autoencoder-based network anomaly detection,” in *2018 Wireless Telecommunications Symposium (WTS)*, 2018, pp. 1–5.
- [15] C. Zhou and R. C. Paffenroth, “Anomaly detection with robust deep autoencoders,” in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’17. New York, NY, USA: Association for Computing Machinery, 2017, p. 665–674. [Online]. Available: <https://doi.org/10.1145/3097983.3098052>
- [16] K. H. Kim, S. Shim, Y. Lim, J. Jeon, J. Choi, B. Kim, and A. S. Yoon, “Rapp: Novelty detection with reconstruction along projection pathway.” in *ICLR*. OpenReview.net, 2020. [Online]. Available: <http://dblp.uni-trier.de/db/conf/iclr/iclr2020.html#KimSLJCKY20>

- [17] J. An and S. Cho, “Variational autoencoder based anomaly detection using reconstruction probability,” 2015.
- [18] S. M. Erfani, S. Rajasegarar, S. Karunasekera, and C. Leckie, “High-dimensional and large-scale anomaly detection using a linear one-class svm with deep learning,” *Pattern Recognition*, vol. 58, pp. 121–134, 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0031320316300267>
- [19] M. Sakurada and T. Yairi, “Anomaly detection using autoencoders with nonlinear dimensionality reduction,” in *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*, ser. MLSDA’14. New York, NY, USA: Association for Computing Machinery, 2014, p. 4–11. [Online]. Available: <https://doi.org/10.1145/2689746.2689747>
- [20] Y. Xia, X. Cao, F. Wen, G. Hua, and J. Sun, “Learning discriminative reconstructions for unsupervised outlier removal,” *2015 IEEE International Conference on Computer Vision (ICCV)*, pp. 1511–1519, 2015.
- [21] D. H. Ballard, “Modular learning in neural networks,” in *Proceedings of the Sixth National Conference on Artificial Intelligence - Volume 1*, ser. AAAI’87. AAAI Press, 1987, p. 279–284.
- [22] L. Ruff, R. Vandermeulen, N. Goernitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft, “Deep one-class classification,” in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, 10–15 Jul 2018, pp. 4393–4402. [Online]. Available: <https://proceedings.mlr.press/v80/ruff18a.html>
- [23] B. Zong, Q. Song, M. R. Min, W. Cheng, C. Lumezanu, D. ki Cho, and H. Chen, “Deep autoencoding gaussian mixture model for unsupervised anomaly detection,” in *International Conference on Learning Representations*, 2018.
- [24] T. Shenkar and L. Wolf, “Anomaly detection for tabular data with internal contrastive learning,” in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=_hszZbt46bT
- [25] G. Steinbuss and K. Böhm, “Benchmarking unsupervised outlier detection with realistic synthetic data,” *ACM Transactions on Knowledge Discovery from Data*, vol. 15, no. 4, pp. 1–20, apr 2021. [Online]. Available: <https://doi.org/10.1145%2F3441453>
- [26] G. W. Milligan, “An algorithm for generating artificial test clusters,” *Psychometrika*, vol. 50, pp. 123–127, 1985.
- [27] J. C. Duchi, “Derivations for linear algebra and optimization,” 2016.
- [28] C. Aggarwal, A. Hinneburg, and D. Keim, “On the surprising behavior of distance metric in high-dimensional space,” *First publ. in: Database theory, ICDT 200, 8th International Conference, London, UK, January 4 - 6, 2001 / Jan Van den Bussche ... (eds.). Berlin: Springer, 2001, pp. 420-434 (=Lecture notes in computer science ; 1973), 02 2002.*
- [29] T. T. Dang, H. Y. Ngan, and W. Liu, “Distance-based k-nearest neighbors outlier detection method in large-scale traffic data,” in *2015 IEEE International Conference on Digital Signal Processing (DSP)*, 2015, pp. 507–510.
- [30] D. Bank, N. Koenigstein, and R. Giryes, “Autoencoders,” 2021.
- [31] W. H. Lopez Pinaya, S. Vieira, R. Garcia-Dias, and A. Mechelli, “Chapter 11 - autoencoders,” in *Machine Learning*, A. Mechelli and S. Vieira, Eds. Academic Press, 2020, pp. 193–208. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/B9780128157398000110>
- [32] E. Plaut, “From principal subspaces to principal components with linear autoencoders,” 2018.
- [33] Q. Xu, Z. Wu, Y. Yang, and L. Zhang, “The difference learning of hidden layer between autoencoder and variational autoencoder,” in *2017 29th Chinese Control And Decision Conference (CCDC)*, 2017, pp. 4801–4804.

- [34] K. O'Shea and R. Nash, "An introduction to convolutional neural networks," 2015.
- [35] Z. Tan, A. Jamdagni, X. He, P. Nanda, and R. Liu, "A system for denial-of-service attack detection based on multivariate correlation analysis," *IEEE transactions on parallel and distributed systems*, no. 2, pp. 447–456, Feb. 2014.
- [36] F. J. Pulgar, F. Charte, A. J. Rivera, and M. J. del Jesus, "Aeknn: An autoencoder knn-based classifier with built-in dimensionality reduction," 2018.
- [37] J. Guo, G. Liu, Y. Zuo, and J. Wu, "An anomaly detection framework based on autoencoder and nearest neighbor," in *2018 15th International Conference on Service Systems and Service Management (ICSSSM)*, 2018, pp. 1–6.